

Universidade Federal de Minas Gerais

Automação em Tempo Real

Sistema de Inspeção Automática de Túneis

Relatório da Etapa 1

Integrante 1: Nome completo – Matrícula 000000

Integrante 2: Nome completo – Matrícula 000000

Integrante 3: Nome completo – Matrícula 000000

Professor: Leandro Freitas

Belo Horizonte
17 de maio de 2026

Sumário

1	Introdução	2
2	Proposta de Arquitetura	3
2.1	Diagrama da Arquitetura	3
2.2	Tarefas Implementadas	3
2.3	Buffers e Estados Compartilhados	4
2.4	Mecanismos de Sincronização	4
3	Implementação Desenvolvida	5
3.1	Tipos de Dados	5
3.2	Buffers Sincronizados	5
3.3	Simulação dos Sensores	6
3.4	Consumo com Variável de Condição	6
3.5	Reconstrução da Superfície	7
3.6	Detecção de Falha	7
3.7	Distância Percorrida	8
3.8	Coletor de Dados	8
3.9	Controle de Navegação	9
3.10	Inspecção por Câmera	9
3.11	Criação das Threads	10
3.12	Compilação	11
4	Resultados de Testes Preliminares	11
4.1	Teste 1: Troca de Dados entre Simulação e Reconstrução	11
4.2	Teste 2: Cálculo de Distância Percorrida	11
4.3	Teste 3: Coletor de Dados e Arquivo CSV	12
4.4	Teste 4: Detecção de Falha e Acionamento da Câmera	12
4.5	Análise dos Testes	12
5	Conclusão	12

1 Introdução

A inspeção periódica de túneis e estruturas civis é uma atividade importante para a segurança pública. Métodos manuais de inspeção podem ser lentos, caros e perigosos, principalmente quando envolvem ambientes de difícil acesso ou risco estrutural. Nesse contexto, o uso de robôs autônomos permite coletar dados de sensores em tempo real e reduzir a exposição de operadores humanos.

O projeto consiste em desenvolver um sistema de inspeção automática de túneis em um cenário simplificado em duas dimensões. O robô se desloca horizontalmente pelo túnel e mede a distância vertical até o teto usando um sensor LIDAR. A partir dessas medições, o sistema reconstrói o perfil da superfície e detecta anomalias, como buracos ou saliências. Quando uma variação brusca é detectada, o sistema reduz a velocidade de navegação e aciona uma tarefa de inspeção detalhada por câmera.

Os objetivos desta primeira etapa são:

- definir a arquitetura completa do sistema;
- implementar as tarefas internas do robô como threads em C++;
- implementar buffers compartilhados entre tarefas;
- utilizar mecanismos de sincronização, como mutexes e variáveis de condição;
- realizar testes preliminares de troca de dados entre tarefas;
- documentar a arquitetura e os mecanismos de sincronização utilizados.

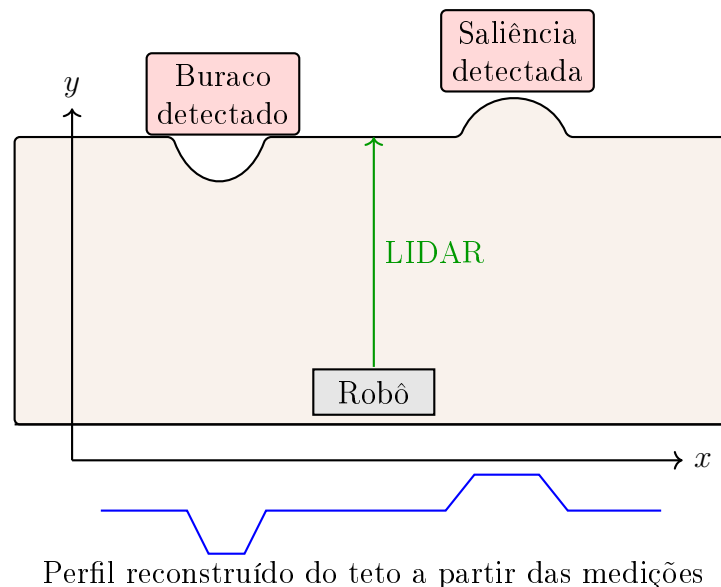


Figura 1: Ilustração própria do problema tratado: robô inspecionando o teto de um túnel em vista lateral 2D.

2 Proposta de Arquitetura

Na Etapa 1 foi adotada uma arquitetura multitarefa em processo único. As tarefas internas do robô, correspondentes aos blocos de controle embarcado, foram implementadas como threads C++ dentro do processo principal. A comunicação entre essas tarefas ocorre por memória compartilhada, com buffers protegidos por `std::mutex` e `std::condition_variable`.

Essa decisão é adequada para a primeira etapa porque permite demonstrar diretamente os conceitos de concorrência, região crítica, produtor-consumidor e sincronização. A comunicação via IPC/MQTT, bem como as interfaces gráficas de simulação e operação remota, ficam planejadas para a Etapa 2.

2.1 Diagrama da Arquitetura

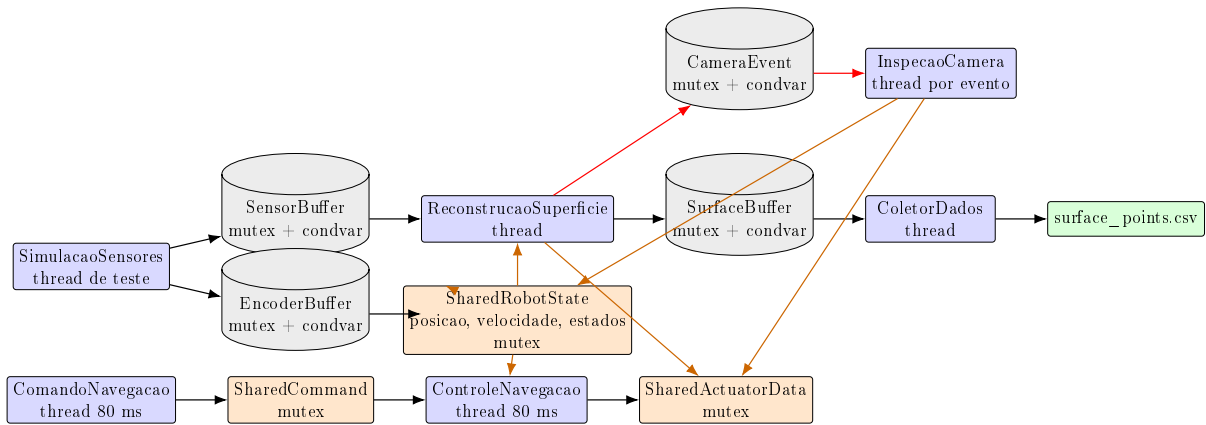


Figura 2: Arquitetura detalhada da Etapa 1, com threads, buffers, estados compartilhados e mecanismos de sincronização.

2.2 Tarefas Implementadas

Tarefa	Tipo	Responsabilidade
SimulacaoSensores	Thread C++ de teste	Gera dados simulados de LIDAR e encoder para testar os buffers da Etapa 1.
ReconstrucaoSuperficie	Thread C++	Consome leituras de LIDAR, aplica média móvel, gera pontos de superfície e detecta anomalias.
DistanciaPercorrida	Thread C++	Consome o encoder e calcula a posição horizontal do robô.
ColetorDados	Thread C++	Consome pontos reconstruídos, calcula confiança online e grava os dados em CSV.
ComandoNavegacao	Thread C++ cíclica	Simula comando de operação remota na Etapa 1, configurando modo automático e setpoint.

Tarefa	Tipo	Responsabilidade
ControleNavegacao	Thread C++ cíclica	Aplica controle proporcional simples e gera aceleração.
InspecaoCamera	Thread C++ por evento	Aguarda sinal de falha e executa processamento pesado usando CPU.

2.3 Buffers e Estados Compartilhados

Os buffers implementam o padrão produtor-consumidor. Cada buffer possui uma fila, um mutex, uma variável de condição e uma flag de finalização.

Estrutura	Fluxo	Sincronização
SensorBuffer	SimulacaoSensores -> ReconstrucaoSuperficie	mutex_sensor e dado_disponivel_var.
EncoderBuffer	SimulacaoSensores -> DistanciaPercorrida	mutex_encoder e encoder_disponivel_var.
SurfaceBuffer	ReconstrucaoSuperficie -> ColetorDados	mutex_superficie e surface_point_var.
CameraEvent	ReconstrucaoSuperficie -> InspecaoCamera	mutex_camera e camera_event_var.

Além dos buffers, foram utilizados estados compartilhados protegidos por mutex:

- **SharedRobotState**: armazena posição, velocidade, modo automático e estado de inspeção.
- **SharedCommand**: armazena comandos simulados para a Etapa 1.
- **SharedActuatorData**: armazena aceleração e comando de câmera.
- **SharedSystemControl**: controla o encerramento das tarefas cíclicas.

2.4 Mecanismos de Sincronização

O `std::mutex` é usado para proteger regiões críticas, como inserção e remoção de dados em filas ou leitura e escrita em estados compartilhados. A regra usada no código foi manter o mutex travado apenas durante o acesso ao dado compartilhado, copiando o valor para uma variável local e realizando o processamento fora da região crítica.

A `std::condition_variable` é usada para evitar espera ocupada. As tarefas consumidoras ficam bloqueadas até que uma tarefa produtora insira um novo dado no buffer e chame `notify_one()`. Essa estratégia reduz uso desnecessário de CPU e evita laços de polling.

3 Implementação Desenvolvida

Esta seção apresenta os principais trechos de código do sistema e a justificativa das decisões tomadas em cada parte da implementação. Os trechos foram escolhidos por representarem os mecanismos centrais do trabalho: definição dos dados, comunicação por buffers, sincronização, reconstrução da superfície, detecção de falhas, controle de navegação e acionamento da câmera.

3.1 Tipos de Dados

O primeiro passo da implementação foi definir estruturas simples para representar as mensagens trocadas entre as tarefas. Essa escolha evita passar vários parâmetros soltos entre threads e deixa explícito qual informação circula em cada buffer. `SensorData` representa a leitura combinada de LIDAR e encoder, `EncoderData` separa o fluxo usado pela tarefa de distância, e `SurfacePoint` representa o ponto reconstruído do teto.

```
1 struct SensorData {
2     bool i_encoder;
3     int i_lidar;
4     double timestamp;
5 };
6
7 struct EncoderData {
8     bool i_encoder;
9     double timestamp;
10 };
11
12 struct SurfacePoint {
13     double timestamp;
14     double x;
15     double y;
16     double confianca;
17 };
```

Separar `EncoderData` de `SensorData` foi importante para evitar que duas tarefas consumidoras retirassem dados da mesma fila. A reconstrução consome o fluxo de sensores, enquanto a distância percorrida consome o fluxo específico do encoder.

3.2 Buffers Sincronizados

Os buffers foram implementados seguindo o padrão produtor-consumidor. Cada buffer possui uma fila, um mutex, uma variável de condição e uma flag de finalização. Essa estrutura foi escolhida porque permite que uma tarefa produza dados em seu próprio ritmo e outra tarefa consuma esses dados sem espera ocupada.

```
1 struct SensorBuffer {
2     std::queue<SensorData> fila_sensor;
3     std::mutex mutex_sensor;
4     std::condition_variable dado_disponivel_var;
5     bool finalizado = false;
6 };
7
8 struct SurfaceBuffer {
9     std::queue<SurfacePoint> fila_superficie;
10    std::mutex mutex_superficie;
```

```

11     std::condition_variable surface_point_var;
12     bool finalizado = false;
13 };

```

O `mutex` protege o acesso à fila. A `condition_variable` permite que a tarefa consumidora durma enquanto não há dados disponíveis. A flag `finalizado` evita que a consumidora fique bloqueada indefinidamente ao final da simulação.

3.3 Simulação dos Sensores

A tarefa de simulação foi usada na Etapa 1 para substituir temporariamente a interface gráfica de simulação que será implementada na Etapa 2. Ela gera leituras falsas, alterna o encoder e injeta uma anomalia no LIDAR para testar a detecção de falhas.

```

1  for (int i = 0; i < 10; i++) {
2      estado_encoder = !estado_encoder;
3
4      SensorData leitura;
5      leitura.i_encoder = estado_encoder;
6      leitura.i_lidar = (i == 5) ? 140 : 100 + i;
7      leitura.timestamp = TempoAtualSegundos();
8
9      EncoderData encoder{leitura.i_encoder, leitura.timestamp};
10
11     {
12         std::lock_guard<std::mutex> trava(encoderBuffer.mutex_encoder);
13         encoderBuffer.fila_encoder.push(encoder);
14     }
15     encoderBuffer.encoder_disponivel_var.notify_one();
16
17     {
18         std::lock_guard<std::mutex> trava(sensorBuffer.mutex_sensor);
19         sensorBuffer.fila_sensor.push(leitura);
20     }
21     sensorBuffer.dado_disponivel_var.notify_one();
22
23     std::this_thread::sleep_for(std::chrono::milliseconds(500));
24 }

```

A leitura anômala `i_lidar = 140` foi colocada para simular uma variação brusca no teto. O uso de `sleep_for` aqui representa o intervalo entre leituras de sensores simulados. Isso é diferente da tarefa de câmera, que não deve ser simulada apenas com `sleep`, pois o enunciado exige processamento pesado.

3.4 Consumo com Variável de Condição

As tarefas consumidoras usam `condition_variable` para aguardar dados. Essa decisão evita que a thread execute um laço verificando a fila repetidamente, o que desperdiçaria CPU.

```

1  SensorData leitura;
2
3  {
4      std::unique_lock<std::mutex> trava(buffer.mutex_sensor);
5
6      buffer.dado_disponivel_var.wait(trava, [&buffer] {

```

```

7         return !buffer.fila_sensor.empty() || buffer.finalizado;
8     });
9
10    if (buffer.fila_sensor.empty() && buffer.finalizado) {
11        break;
12    }
13
14    leitura = buffer.fila_sensor.front();
15    buffer.fila_sensor.pop();
16 }

```

O processamento é feito fora da região crítica. Assim, o mutex fica travado apenas pelo tempo necessário para retirar o dado da fila, reduzindo contenção entre threads.

3.5 Reconstrução da Superfície

A reconstrução aplica uma média móvel de três amostras no LIDAR. Essa técnica foi escolhida por ser simples, online e suficiente para reduzir ruídos iniciais sem exigir armazenamento de todo o histórico.

```

1  ultimasLeituras.push(leitura.i_lidar);
2  somaLeituras += leitura.i_lidar;
3
4  if (ultimasLeituras.size() > tamanhoJanela) {
5      somaLeituras -= ultimasLeituras.front();
6      ultimasLeituras.pop();
7  }
8
9  double media_movel =
10      static_cast<double>(somaLeituras) / ultimasLeituras.size();
11
12  SurfacePoint ponto;
13  ponto.timestamp = leitura.timestamp;
14
15  {
16      std::lock_guard<std::mutex> trava(robotState.mutex_estado);
17      ponto.x = robotState.estado.posicao_x;
18  }
19
20  ponto.y = media_movel;
21  ponto.confianca = 1.0;

```

A coordenada x é lida de `SharedRobotState`, atualizado pela tarefa de distância percorrida. Dessa forma, o ponto reconstruído combina posição horizontal e distância vertical filtrada.

3.6 Detecção de Falha

A falha é detectada comparando a média móvel atual com a média anterior. A regra foi escolhida por ser simples de testar e por representar a ideia central do problema: buracos e saliências aparecem como variações bruscas no perfil do teto.

```

1  const double variacao = std::abs(media_movel - mediaAnterior);
2
3  if (variacao > limiteFalha && !falhaJaDetectada) {
4      falhaJaDetectada = true;

```



```

5
6     {
7         std::lock_guard<std::mutex> trava(robotState.mutex_estado);
8         robotState.estado.e_inspecao = true;
9     }
10
11     {
12         std::lock_guard<std::mutex> trava(sharedActuatorData.
13             mutex_atuadores);
14         sharedActuatorData.atuadores.o_liga_camera = true;
15     }
16
17     {
18         std::lock_guard<std::mutex> trava(cameraEvent.mutex_camera);
19         cameraEvent.falha_detectada = true;
20         cameraEvent.timestamp = ponto.timestamp;
21         cameraEvent.x = ponto.x;
22         cameraEvent.y = ponto.y;
23     }
24
25     cameraEvent.camera_event_var.notify_one();
26 }

```

Quando a falha é encontrada, a reconstrução atualiza o estado do robô, liga a câmera e sinaliza o evento que acorda a tarefa *InspecaoCamera*. A flag *falhaJaDetectada* evita múltiplos disparos para a mesma anomalia simulada.

3.7 Distância Percorrida

A tarefa *DistanciaPercorrida* implementa a regra do enunciado: o encoder troca de estado a cada metro percorrido. Sempre que o valor atual do encoder difere do anterior, a posição horizontal é incrementada.

```

1  if (leitura.i_encoder != encoderAnterior) {
2      double posicaoAtualizada = 0.0;
3
4      {
5          std::lock_guard<std::mutex> trava(robotState.mutex_estado);
6          robotState.estado.posicao_x += 1.0;
7          posicaoAtualizada = robotState.estado.posicao_x;
8      }
9
10     encoderAnterior = leitura.i_encoder;
11 }

```

O estado do robô foi centralizado em *SharedRobotState* para que a posição possa ser usada por outras tarefas, como a reconstrução da superfície.

3.8 Coletor de Dados

O coletor recebe os pontos reconstruídos e grava o arquivo CSV. Além disso, calcula uma confiança simples com base na quantidade de pontos próximos já observados. Essa escolha atende ao requisito de análise online, mantendo a implementação simples para a Etapa 1.

```

1  int pontosProximos = 1;
2

```

```

3 for (const SurfacePoint &anterior : historico) {
4     const bool proximoEmX = std::abs(anterior.x - ponto.x) <= 2.0;
5     const bool proximoEmY = std::abs(anterior.y - ponto.y) <= 15.0;
6
7     if (proximoEmX && proximoEmY) {
8         pontosProximos++;
9     }
10 }
11
12 ponto.confianca = std::min(1.0, pontosProximos / 5.0);
13 historico.push_back(ponto);
14
15 arquivo << ponto.timestamp << ", "
16         << ponto.x << ", "
17         << ponto.y << ", "
18         << ponto.confianca << "\n";

```

O CSV permite verificar o perfil reconstruído e serve como evidência objetiva dos testes realizados.

3.9 Controle de Navegação

O controle de navegação foi implementado inicialmente como controlador proporcional. Essa escolha é suficiente para demonstrar a interação entre comando, estado do robô e atuadores, sem introduzir a complexidade completa de um PID na Etapa 1.

```

1 const double setpointVelocidade =
2     emInspecao ? std::min(comando.j_sp_velocidade, 1)
3                 : comando.j_sp_velocidade;
4
5 const double erro = setpointVelocidade - velocidadeAtual;
6 const int aceleracao =
7     static_cast<int>(std::clamp(kp * erro, -100.0, 100.0));
8
9 {
10     std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores);
11     sharedActuatorData.atuadores.o_aceleracao = aceleracao;
12     sharedActuatorData.atuadores.o_liga_camera = emInspecao;
13 }

```

Quando o robô entra em inspeção, o setpoint é limitado para reduzir a velocidade. Essa decisão acompanha o comportamento pedido no enunciado: ao detectar falha, o robô deve navegar mais devagar e acionar a câmera.

3.10 Inspeção por Câmera

A tarefa de câmera é acionada por evento. Ao acordar, ela executa um laço computacional com funções matemáticas, simulando processamento pesado de imagem. Essa escolha foi feita porque o enunciado especifica que essa tarefa deve usar o processador e não apenas dormir.

```

1 cameraEvent.camera_event_var.wait(trava, [&cameraEvent] {
2     return cameraEvent.falha_detectada || cameraEvent.finalizado;
3 });
4

```

```

5 volatile double resultado = 0.0;
6 for (int i = 0; i < 8000000; i++) {
7     resultado += std::sin(i * 0.001) * std::cos(i * 0.0005);
8 }
9
10 {
11     std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores);
12     sharedActuatorData.atuadores.o_liga_camera = false;
13 }

```

O uso de evento evita que a câmera execute continuamente. Ela permanece bloqueada e só consome CPU quando uma falha é detectada.

3.11 Criação das Threads

O programa principal instancia todos os buffers e estados compartilhados, cria as threads e aguarda sua finalização. Essa organização deixa o `main` responsável apenas pela montagem do sistema, mantendo a lógica de cada tarefa em seu próprio arquivo.

```

1 SensorBuffer sensorBuffer;
2 EncoderBuffer encoderBuffer;
3 SurfaceBuffer surfaceBuffer;
4 CameraEvent cameraEvent;
5 SharedRobotState robotState;
6 SharedCommand sharedCommand;
7 SharedActuatorData sharedActuatorData;
8 SharedSystemControl systemControl;
9
10 std::thread comando(ComandoNavegacao,
11     std::ref(sharedCommand), std::ref(robotState), std::ref(
12         systemControl));
13
14 std::thread controle(ControleNavegacao,
15     std::ref(sharedCommand), std::ref(robotState),
16     std::ref(sharedActuatorData), std::ref(systemControl));
17
18 std::thread simulacao(SimulacaoSensores,
19     std::ref(sensorBuffer), std::ref(encoderBuffer));
20
21 std::thread distancia(DistanciaPercorrida,
22     std::ref(encoderBuffer), std::ref(robotState));
23
24 std::thread reconstrucao(ReconstrucaoSuperficie,
25     std::ref(sensorBuffer), std::ref(surfaceBuffer),
26     std::ref(robotState), std::ref(sharedActuatorData), std::ref(
27         cameraEvent));
28
29 std::thread coletor(ColetorDados, std::ref(surfaceBuffer));
30 std::thread camera(InspecaoCamera,
31     std::ref(cameraEvent), std::ref(robotState), std::ref(
32         sharedActuatorData));

```

3.12 Compilação

A compilação foi automatizada por um `Makefile`. Essa escolha reduz erros na apresentação, facilita os testes e garante que todos os arquivos da pasta `src/` sejam compilados com as mesmas opções.

```
CXX := g++
CXXFLAGS := -std=c++17 -Wall -Wextra -pthread -Iinclude
BUILD_DIR := build
TARGET := $(BUILD_DIR)/rt_inspection
SRCS := $(wildcard src/*.cpp)

all: $(TARGET)

$(TARGET): $(SRCS)
    mkdir -p $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)

run: $(TARGET)
    ./$(TARGET)
```

4 Resultados de Testes Preliminares

Os testes foram executados com os seguintes comandos:

```
make clean
make
make run
```

4.1 Teste 1: Troca de Dados entre Simulação e Reconstrução

O primeiro teste verificou se a tarefa `SimulacaoSensores` produz leituras de sensores e se a tarefa `ReconstrucaoSuperficie` consome essas leituras corretamente. O resultado esperado é a impressão de leituras com o valor filtrado por média móvel.

```
Reconstrucao: encoder=1 lidar=100 media_movel=100 timestamp=...
Reconstrucao: encoder=0 lidar=101 media_movel=100.5 timestamp=...
Reconstrucao: encoder=1 lidar=102 media_movel=101 timestamp=...
```

Esse teste valida o funcionamento do `SensorBuffer`, do mutex associado e da variável de condição usada para acordar a reconstrução.

4.2 Teste 2: Cálculo de Distância Percorrida

O segundo teste verificou se o encoder é processado pela tarefa `DistanciaPercorrida`. Como o encoder simulado troca de estado a cada leitura, a posição horizontal é incrementada quando ocorre uma mudança.

```
DistanciaPercorrida: x=1 m timestamp=...
DistanciaPercorrida: x=2 m timestamp=...
DistanciaPercorrida: x=3 m timestamp=...
```

Esse teste valida o `EncoderBuffer` e o uso de `SharedRobotState` para compartilhar a posição com outras tarefas.

4.3 Teste 3: Coletor de Dados e Arquivo CSV

O terceiro teste verificou se a reconstrução gera pontos de superfície e se o coletor consome esses pontos, calcula confiança e grava o arquivo `surface_points.csv`.

```
timestamp,x,y,confianca
1.886e-06,0,100,0.2
0.500124,0,100.5,0.4
1.00024,2,101,0.6
2.50063,4,115.667,0.8
```

Esse teste valida o `SurfaceBuffer`, o cálculo online de confiança e a gravação dos dados em disco.

4.4 Teste 4: Detecção de Falha e Acionamento da Câmera

O quarto teste inseriu uma anomalia no LIDAR simulado, configurando uma leitura com valor mais alto que o padrão. A reconstrução compara a média móvel atual com a média anterior e sinaliza falha quando a variação passa do limite configurado.

```
Falha detectada: x=4 y=115.667 variacao=12.6667 limite=10
InspecaoCamera: iniciando inspecao em x=4 y=115.667 timestamp=...
InspecaoCamera: processamento finalizado resultado=...
```

Esse teste valida o `CameraEvent`, a sincronização por evento e o processamento pesado da tarefa de câmera. A câmera executa um laço com funções matemáticas, usando CPU de fato, e não apenas uma chamada de `sleep`.

4.5 Análise dos Testes

Durante os testes, as threads finalizaram corretamente, sem bloqueio observado. Os buffers possuem flags de finalização para evitar que consumidores fiquem bloqueados indefinidamente. As impressões no terminal são protegidas por um mutex global de log, evitando mensagens misturadas. A troca de dados entre tarefas ocorreu de forma coerente, com produção, consumo e encerramento ordenados.

5 Conclusão

Nesta etapa foi implementado o núcleo embarcado do sistema de inspeção automática de túneis. As principais tarefas foram modeladas como threads C++ dentro de um único processo. Foram implementados buffers sincronizados para comunicação entre tarefas, estados compartilhados protegidos por mutex, reconstrução do perfil do teto por média móvel, cálculo de distância percorrida pelo encoder, controle proporcional simples, detecção de falha por variação brusca e acionamento de inspeção por câmera via evento.

Os testes preliminares indicam que a arquitetura implementada consegue trocar dados entre tarefas sem condições de corrida aparentes, sem bloqueios observados e com finalização controlada das threads. O sistema também gera um arquivo CSV com os pontos reconstruídos, permitindo verificar o perfil estimado do teto.

Como próximos passos para a Etapa 2, devem ser implementadas as interfaces gráficas de simulação e operação remota, a comunicação entre processos via MQTT, a integração com um broker publisher/subscriber e uma análise mais detalhada de tempo de resposta e escalonabilidade das tarefas cíclicas.